



UNIVERSIDADE ESTADUAL DE SANTA CRUZ
DEPARTAMENTO DE CIÊNCIAS EXATAS E TECNOLÓGICAS
CIÊNCIA DA COMPUTAÇÃO

ANA LUIZA OLIVEIRA DE FREITAS

YURI COUTINHO COSTA

BENCHMARK DE SUAVIZAÇÃO COM MPI

Ilhéus - BA

2026

ANA LUIZA OLIVEIRA DE FREITAS

YURI COUTINHO COSTA

BENCHMARK DE SUAVIZAÇÃO COM MPI

Relatório apresentado à Faculdade de Ciência da Computação da Universidade Estadual Santa Cruz como requisito parcial para a conclusão da disciplina de Processamento Paralelo, no semestre de 2026.1.

Orientador (a): Prof. Dr. Esbel Tomás Valero Orellana.

Ilhéus - BA

2026

1. INTRODUÇÃO

1.1 Objetivo

Otimizar um algoritmo de suavização/desfoque Gaussiano iterativo em imagens 2D, aplicando técnicas de paralelismo em memória distribuída com MPI.

1.2 Conceitos Essenciais

1.2.1 Suavização/Desfoque Gaussiano

A suavização gaussiana é uma técnica de processamento de imagens que atua como um filtro passa-baixa, reduzindo ruídos e suavizando detalhes por meio de uma média ponderada baseada na distribuição normal. O processo utiliza um kernel quadrado de dimensão ímpar, cujos pesos são ditados pelo desvio padrão/sigma. Enquanto o tamanho do kernel define a área de vizinhança que influenciará o pixel central, o sigma controla a dispersão da "curva de sino" gaussiana; quanto maior o valor de sigma, maior será o peso dado aos pixels vizinhos e, conseqüentemente, mais intenso será o efeito de borrão.

Cada elemento do kernel segue uma função exponencial e, em seguida, é aplicada a normalização para que a soma total de seus pesos seja igual a 1, evitando alterações indesejadas no brilho da imagem. A aplicação prática ocorre via convolução, onde o kernel desliza sobre a imagem original para calcular o novo valor de cada pixel.

1.2.2 Paralelismo de memória distribuída e MPI

O paralelismo de memória distribuída, também conhecido como multiprocessamento, se refere à execução de tarefas como múltiplos processos que não compartilham o mesmo espaço de memória. Pela independência de memória dos processos, é necessário que, em algum momento, eles se comuniquem para a conclusão dos cálculos, o que pode resultar em um alto nível de comunicação em casos mais complexos.

O método mais comum para implementar paralelismo de memória distribuída é o *Message Passing Interface* (Interface de Passagem de Mensagens) ou MPI.

O MPI é um padrão de biblioteca definido pelo MPI Fórum em 1994, utilizado na programação paralela para dividir e distribuir tarefas entre múltiplos processos ou computadores, comunicando-se através do envio e recebimento de mensagens. Atualmente na versão 5.0, MPI-5, possui mais de 125 funções para programação e pode ser implementado em C, C++ e Fortran. Além disso, utiliza predominantemente o modelo de programação SPMD (*Single Program - Multiple Data*), ou seja, um único código-fonte é compilado e executado, mas múltiplas instâncias desse programa são iniciadas simultaneamente em diferentes processadores ou nós.

1.2.3 Decomposição de Domínios e Troca de Halos

Na implementação do paralelismo distribuído é necessário decompor um programa em pequenas tarefas que serão executadas em paralelo, e, para isso, é preciso utilizar a decomposição de domínio. A partir dessa decomposição é possível realizar a divisão de um volume de dados do problema em partes menores ou grupos que serão distribuídos entre os múltiplos processos que executarão, simultaneamente, um mesmo programa.

Tendo em vista que o Desfoque Gaussiano é um algoritmo de vizinhança, é preciso que, após a divisão do domínio, cada processo MPI tenha acesso às bordas/halos dos processos vizinhos para que seja possível realizar o cálculo da própria região. Por esse motivo, os processos trocam frequentemente os valores atualizados dessas bordas, aumentando consequentemente a quantidade de comunicações entre processos.

1.3 Importância da análise de desempenho em sistemas distribuídos

A análise de desempenho é a etapa que valida a eficácia da paralelização, transformando dados de tempo em métricas quantificáveis. Em processamento paralelo, essa análise permite determinar o *Speedup*, que mede o ganho de velocidade obtido ao distribuir a carga de trabalho entre os múltiplos processos em relação à execução puramente sequencial e a *Eficiência* do sistema, indicando o quão bem os recursos do hardware, como os núcleos de

performance e processador, estão sendo aproveitados para resolver o problema proposto. Em sistemas distribuídos, a análise de desempenho se torna ainda mais necessária para identificação de gargalos, tendo em vista a necessidade de evitar que os processos fiquem boa parte do tempo ociosos e aumentem o tempo de execução. Além disso, essa análise pode ser uma forma de garantir a escalabilidade do sistema, permitindo avaliar se a arquitetura respeita os limites de ganho de velocidade impostos pela Lei de Amdahl.

2. METODOLOGIA

2.1 Descrição da implementação paralela com MPI (quais funções MPI foram usadas, como foi feita a decomposição de domínio e a troca de halos)

2.1.1. Funções MPI

Na implementação foram utilizadas as funções *MPI_Init* e *MPI_Finalize* que são utilizadas para inicialização e encerramento do ambiente de execução MPI, respectivamente. Além disso, para definir a estrutura da execução, foram utilizadas as funções *MPI_Comm_rank* que retorna o identificador de cada processo e a *MPI_Comm_size* que retorna a quantidade de processos associados a um comunicador.

Já na implementação da lógica do paralelismo com MPI, foram utilizadas as funções:

- *MPI_Bcast*: utilizado para transmitir uma mensagem a partir de um único processo para todos os processos do mesmo comunicador.
- *MPI_Scatter*: divide e distribui pedaços com tamanhos iguais de um vetor de dados do processo raiz para os demais processos no mesmo comunicador.
- *MPI_Sendrecv*: permite envio e recebimento de mensagens simultaneamente, em buffers distintos. Os buffers podem ser de tamanhos e tipos diferentes.
- *MPI_Gather*: une os dados de todos os processos de um comunicador e envia para o processo raiz.

2.1.2. Decomposição de Domínio

Na primeira versão da implementação com MPI, foi utilizada a decomposição de domínios por linhas a partir da função `MPI_Scatter`. Nesta, é realizada a divisão do número de linhas da matriz pelo número de processos utilizados na execução do programa, assim, considerando os tamanhos de imagem testados, é possível ter certeza que os processos terão fatias de mesmo tamanho.

2.1.3 Troca de Halos

Para o cálculo dos pixels das bordas utilizamos a função `MPI_Sendrecv`, que gerencia o envio e recebimento de dados de mesmo tamanho. Através de dois *ifs*, controlamos a quantidade de recebimento e envio de dados.

- a. Caso os processos não estejam em uma das bordas, vão enviar duas linhas de pixels e receber duas linhas de pixels;
- b. Caso os processos estejam nas bordas, (Rank0 e Rank3/Rank7) vão enviar uma linha e receber uma linha.

2.2 Descrição do hardware utilizado nos testes

MacBook Air

1. Processador: Apple M5
2. CPU de 10 núcleos (4 supernúcleos e 6 núcleos de eficiência)
3. Sistema Operacional: macOS
4. Compilador: GCC 15.2.0

Compilação: `mpicc -O3 filtro_gauss_MPI.c -o filtro`

Execução: `mpirun -np 4 ./filtro`

2.4 Procedimentos para medição de tempo

Na medição do tempo, foi utilizada a função `MPI_Wtime()` que retorna o *wall clock time*, ou seja, o tempo total decorrido entre o início e o fim de uma tarefa paralela, decorrido em segundos como um valor de precisão dupla.

2.5 Parâmetros utilizados nos testes de desempenho

Variável	Valores Testados
Resoluções das Imagens	512px, 1024px, 2048px, 4096px
Kernel	3x3, 5x5
Número de iterações	100,500,1000
Número de processos	1,2,4,8

Cada teste foi realizado 5 vezes, totalizando 480 testes realizados.

2.6 Métricas utilizadas para avaliação

- Tempo de Execução: O tempo total gasto para processar a imagem;
- Speedup: A razão entre o tempo de execução sequencial e o tempo de execução paralelo. Indica o ganho de velocidade;
- Eficiência: O Speedup dividido pelo número de processadores/threads. Indica o quão bem os recursos paralelos estão sendo utilizados;
- Fator de Balanceamento de Carga: Quando aplicável (e.g., em MPI com dados desiguais), indica a distribuição do trabalho entre os elementos de processamento;
- Discussão sobre Escalabilidade: Análise de como o desempenho se comporta ao aumentar o número de processadores/threads ou o tamanho do problema.

3. RESULTADOS

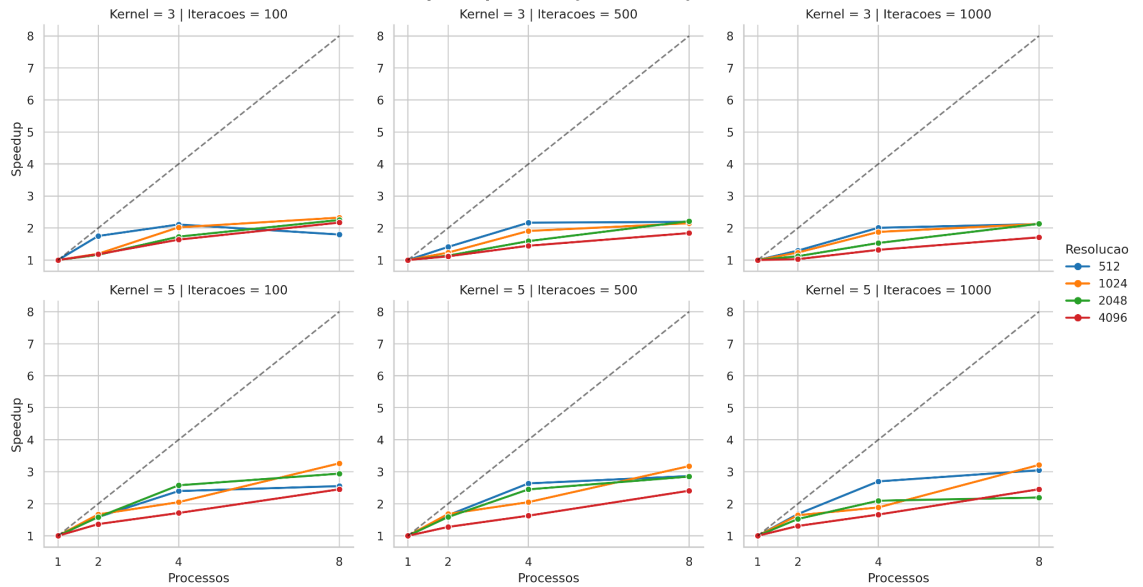
3.1 Teste de Corretude

Primeiro utilizamos o código em python com a matriz simples e os parâmetros de exemplo e comparamos com o resultado gerado pelo código serial em C, assim foi possível verificar que os resultados eram idênticos. Em seguida, elaboramos um *shell script* para executar o código serial e paralelo com diferentes resoluções, tamanho de kernel, quantidade de iterações e número de processos. Ao mesmo tempo, comparamos a saída da imagem gerada por 8 processos com a gerada pelo código serial, utilizando os mesmos parâmetros. Essa comparação foi feita utilizando o *compare* do *imagemagick* tolerando um erro de 1% e com o comando *cmp* que não tolera erros. Dessa forma, foi

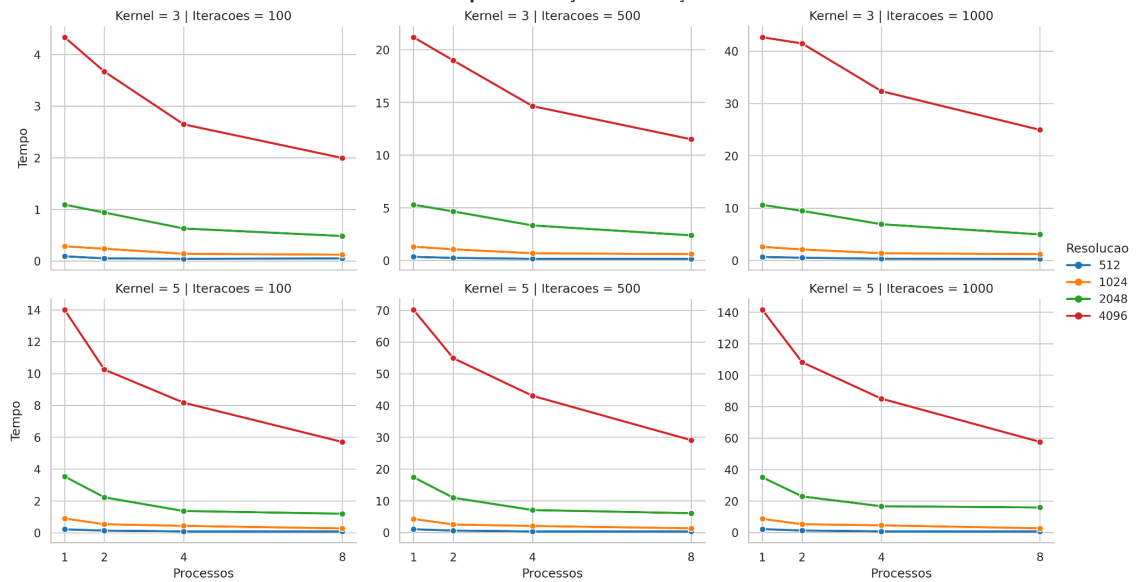
possível verificar que não há diferença nos resultados produzidos, já que, em ambos, foi constatado que as imagens eram idênticas.

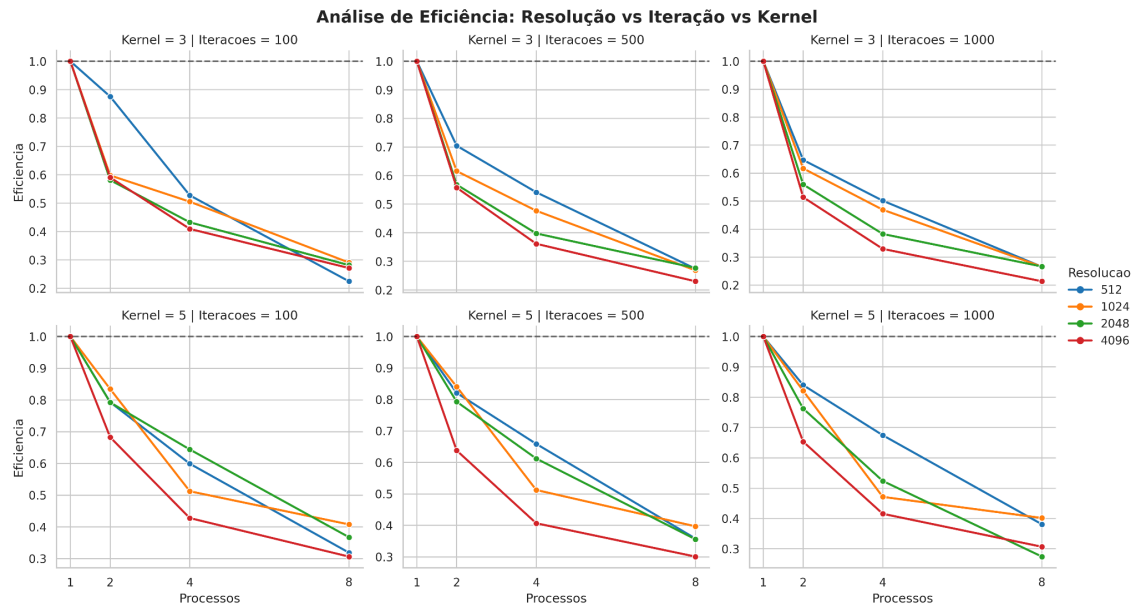
3.2 Gráficos e/ou Tabelas

Análise de Speedup: Resolução vs Iteração vs Kernel



Análise de Tempo: Resolução vs Iteração vs Kernel





3.3 Cálculos de Speedup e Eficiência

Resolução	Iterações	Processos	Kernel	Tempo (s)	T Seq (s)	Speedup	Eficiência
512	100	2	3	0.05308	0.0929	1.7512	0.8756
512	100	2	5	0.140762	0.2233	1.5865740	0.79328701
						28028398	4014199
512	100	4	3	0.0440	0.0929	2.1099046	0.52747615
						261535874	65383969
512	100	4	5	0.0931	0.2233	2.3974670	0.59936677
						843343837	10835959
512	100	8	3	0.0517	0.0929	1.7961279	0.22451598
						103468266	879335333
512	100	8	5	0.0875106	0.2233	2.5520260	0.31900325
						783646025	97955753
512	500	2	3	0.250629	0.3531	1.4090643	0.70453219
512	500	2	5	0.6594824	1.0828	939847345	69923672
						1.6419952	0.82099764
512	500	2	5	0.6594824	1.0828	981307764	90653882

512	500	4	3	0.162964	0.3531524 000000000 3	2.1670577 55087013	0.54176443 87717533
512	500	4	5	0.4107467 999999999 7	1.082867	2.6363370 32936106	0.65908425 82340265
512	500	8	3	0.1610316	0.3531524 000000000 3	2.1930627 28060828	0.27413284 10076035
512	500	8	5	0.3777678	1.082867	2.8664883 560748167	0.35831104 45093521
512	1000	2	3	0.5326122	0.6894663 999999999	1.2944998 255766578	0.64724991 27883289
512	1000	2	5	1.3307541 999999999	2.2356123 333333335	1.6799588 784565427	0.83997943 92282713
512	1000	4	3	0.3435446	0.6894663 999999999	2.0069196 255740884	0.50172990 63935221
512	1000	4	5	0.8283562	2.2356123 333333335	2.6988538 666497983	0.67471346 66624496
512	1000	8	3	0.3245969 999999999 7	0.6894663 999999999	2.1240689 223868365	0.26550861 529835457
512	1000	8	5	0.7336178	2.2356123 333333335	3.0473801 662573257	0.38092252 07821657
1024	100	2	3	0.2392754	0.285997	1.1952628 644649639	0.59763143 22324819
1024	100	2	5	0.5417288	0.904308	1.6693002 107327504	0.83465010 53663752
1024	100	4	3	0.1415634	0.285997	2.0202750 14587104	0.50506875 3646776
1024	100	4	5	0.4410714 000000000 6	0.904308	2.0502530 882755035	0.51256327 20688759
1024	100	8	3	0.1230204	0.285997	2.3247932 86316741	0.29059916 078959264

1024	100	8	5	0.2770620 000000000 3	0.904308	3.2639192 671676374	0.40798990 839595467
1024	500	2	3	1.0651454	1.311827	1.2315942 968912976	0.61579714 84456488
1024	500	2	5	2.5773652	4.3343666 66666667	1.6817045 045330272	0.84085225 22665136
1024	500	4	3	0.6874538	1.311827	1.9082402 337437077	0.47706005 84359269
1024	500	4	5	2.113492	4.3343666 66666667	2.0508081 72761793	0.51270204 31904482
1024	500	8	3	0.6106132	1.311827	2.1483764 18983409	0.26854705 237292614
1024	500	8	5	1.364274	4.3343666 66666667	3.1770499 669909906	0.39713124 587387383
1024	1000	2	3	2.1158045 999999997	2.6114068	1.2342381 711430255	0.61711908 55715127
1024	1000	2	5	5.3305346	8.7554973 33333333	1.6425176 81684935	0.82125884 08424675
1024	1000	4	3	1.3906152	2.6114068	1.8778787 97815528	0.46946969 9453882
1024	1000	4	5	4.6433285 99999999	8.7554973 33333333	1.8856079 523067426	0.47140198 807668565
1024	1000	8	3	1.2275411 999999999	2.6114068	2.1273475 790466345	0.26591844 73808293
1024	1000	8	5	2.7243488	8.7554973 33333333	3.2137945 527875624	0.40172431 90984453
2048	100	2	3	0.9394727 999999999	1.0919744	1.1623267 858313728	0.58116339 29156864
2048	100	2	5	2.2321478	3.5381929 999999997	1.5851069 539391611	0.79255347 69695806
2048	100	4	3	0.6312166	1.0919744	1.7299519 689437826	0.43248799 223594564
2048	100	4	5	1.3730982	3.5381929 999999997	2.5767953 085948254	0.64419882 71487064

2048	100	8	3	0.4850222 000000000 7	1.0919744	2.2513905 549065587	0.28142381 936331984
2048	100	8	5	1.2026822	3.5381929 999999997	2.9419184 88525065	0.36773981 106563314
2048	500	2	3	4.6623016	5.299037	1.1365710 446531387	0.56828552 23265694
2048	500	2	5	11.009047	17.461015 333333332	1.5860605 675798578	0.79303028 37899289
2048	500	4	3	3.3310451 999999997	5.299037	1.5908030 908736996	0.39770077 27184249
2048	500	4	5	7.1276192	17.461015 333333332	2.4497682 667072525	0.61244206 66768131
2048	500	8	3	2.392837	5.299037	2.2145415 671857296	0.27681769 58982162
2048	500	8	5	6.1278394	17.461015 333333332	2.8494570 750880532	0.35618213 438600665
2048	1000	2	3	9.4940798	10.628561 999999999	1.1194936 448711963	0.55974682 24355981
2048	1000	2	5	23.028680 400000002	35.125159 66666667	1.5252788 721088275	0.76263943 60544138
2048	1000	4	3	6.9403678	10.628561 999999999	1.5314119 231548506	0.38285298 078871266
2048	1000	4	5	16.763281 6	35.125159 66666667	2.0953629 78730052	0.52384074 4682513
2048	1000	8	3	4.9868404	10.628561 999999999	2.1313218 686525435	0.26641523 358156793
2048	1000	8	5	16.003854	35.125159 66666667	2.1947938 082081144	0.27434922 60260143
4096	100	2	3	3.667186	4.3313276	1.1811038 763782364	0.59055193 81891182
4096	100	2	5	10.253307 2	14.000871 333333334	1.3654980 837142316	0.68274904 18571158
4096	100	4	3	2.6459204	4.3313276	1.6369833 34797222	0.40924583 36993055

4096	100	4	5	8.1768738	14.000871 333333334	1.7122523 44329117	0.42806308 60822793
4096	100	8	3	1.9956922	4.3313276	2.1703384 92078087	0.27129231 150976085
4096	100	8	5	5.7092019 99999999	14.000871 333333334	2.4523342 024565493	0.30654177 530706866
4096	500	2	3	18.999673 6	21.198224 2	1.1157151 773386254	0.55785758 86693127
4096	500	2	5	54.959659 4	70.148483 33333333	1.2763631 37966123	0.63818156 89830616
4096	500	4	3	14.659175 8	21.198224 2	1.4460720 363282633	0.36151800 90820658
4096	500	4	5	43.096529 999999994	70.148483 33333333	1.6277060 66667858	0.40692651 66669645
4096	500	8	3	11.509574 6	21.198224 2	1.8417904 168239196	0.23022380 210298996
4096	500	8	5	29.133039 000000004	70.148483 33333333	2.4078670 039652685	0.30098337 549565857
4096	1000	2	3	41.491311	42.674885 800000006	1.0285258 472551038	0.51426292 36275519
4096	1000	2	5	108.30775 52	141.55512 6	1.3069712 850996287	0.65348564 25498143
4096	1000	4	3	32.372737 4	42.674885 800000006	1.3182353 185863116	0.32955882 96465779
4096	1000	4	5	85.146077 6	141.55512 6	1.6624973 221314896	0.41562433 05328724
4096	1000	8	3	24.970416 399999998	42.674885 800000006	1.7090177 879452586	0.21362722 349315733
4096	1000	8	5	57.672119 599999995	141.55512 6	2.4544810 73034812	0.30681013 41293515

3.4 Balanceamento de carga e escalabilidade

O desbalanceamento de carga ocorre principalmente durante a troca de informações entre os vizinhos no *MPI_SendRecv*. Os processos do meio executam os dois blocos *if*, enquanto os processos das bordas enviam e recebem

apenas uma linha de pixels. Com isso, os ranks das pontas terminam a etapa de sincronização de halos mais rápido que os outros, e consequentemente, ficam ociosos esperando o fim das comunicações dos processos centrais.

Com relação à escalabilidade, os resultados dos testes reforçaram a alta dependência entre o SpeedUp e o tamanho do problema. Tal aspecto ocorre principalmente em resoluções mais baixas em que o aumento do número de processos afeta o número de comunicações na função *MPI_Sendrecv*, o que gera uma grande diminuição na eficiência.

4. DISCUSSÃO

4.1 Análise da corretude das implementações

Como nosso teste de corretude não gerou nenhum erro e/ou imagem diferente, podemos concluir que a implementação paralela está correta. A equivalência bit a bit (via *cmp*) e a validação visual (via *compare*) confirmam que o particionamento do domínio não introduziu condições de corrida ou erros de sincronização na escrita da imagem de saída.

4.2 Comparação entre a versão sequencial e a paralela com MPI

As versões paralelas apresentaram uma redução drástica no tempo de resposta em relação à execução sequencial, especialmente em resoluções mais altas, com grande número de iterações e maior tamanho do kernel. Tal fato é justificado pelo custo de coordenar processos e abrir canais de comunicação que acaba sendo maior que o tempo que um único processo leva para computar os pixels.

Por esse motivo, observamos que com a Resolução 4096x4096, 1000 iterações, kernel 5x5 com 8 processos, a versão paralela foi 2.45 vezes mais rápida que a sequencial, com 57.67 segundos contra 141.55 segundos.

Entretanto, com a análise estatística dos dados, foi possível perceber que, na média geral dos testes, a resolução 512 x 512 obteve o melhor Speedup médio e a a resolução 1024 x 1024 teve o melhor SpeedUp registrado em um dos testes, sendo 3.26 vezes mais rápida que a versão sequencial. Isto evidencia que, embora as grandes resoluções apresentem o maior ganho em tempo absoluto poupado, elas sofrem mais com o overhead de E/S e com o limite de largura de banda da memória RAM ao movimentar matrizes maiores, enquanto

as resoluções menores tiram um proveito mais constante das memórias Cache da CPU.

Já no quesito eficiência, o melhor cenário de aproveitamento de hardware ocorreu na configuração com Resolução 512x512, 100 iterações, kernel 3x3 com 2 processos, com eficiência de 87,56%.

4.3 Impacto do aumento do número de processos

Foi analisado que, de forma geral, o aumento do número de processos gera uma diminuição no tempo de execução, já que a imagem é dividida em pedaços menores e, assim, cada processo consegue realizar os cálculos de forma mais rápida. No caso com Resolução 4096x4096, 1000 iterações, Kernel 5x5 é possível observar essa discrepância de resultados com maior clareza.

- 2 Processos: 108.30 segundo;
- 4 Processos: 85.14 segundos;
- 8 Processos: 57.67 segundos.

Apesar da redução do tempo, o aumento da quantidade de processos gera uma maior necessidade de troca de halos, ou seja, mais comunicações entre processos, o que impacta negativamente a Eficiência. Analisando o mesmo caso abordado anteriormente, temos que:

- 2 Processos: Eficiência de 65,34%;
- 4 Processos: Eficiência cai para 41,56%;
- 8 Processos: Eficiência despenca para 30,68%.

4.4 Discussão de gargalos e limitações encontradas (como overhead de comunicação)

4.4.1. Distribuição das partes entre os processos

Tendo em vista a necessidade de divisão de domínio entre os processos e as dimensões das imagens de teste, utilizamos a função MPI_Scatter. Entretanto, nos casos em que as dimensões da imagem não são divisíveis pelo número de processos que serão utilizados, seria necessário a utilização do MPI_Scatterv que distribui tamanhos variados de dados. Mas, com essa divisão desigual, os processos com menor

quantidade de pixels terminariam mais rápido e ficariam ociosos, gerando um desbalanceamento de carga.

4.4.2. Limitação de Entrada e Saída

A leitura e a escrita no arquivo são realizadas pelo rank 0, enquanto os outros processos estão em modo espera. Essa etapa representa a fração de código que não pode ser paralelizada, e, portanto, representa a limitação designada pela Lei de Amdhal.

4.4.3. Overhead de comunicação

À medida que foi aumentando o número de processos, foi constatado que a eficiência caiu. Isso ocorre pois existe um overhead na comunicação e ao aumentar a quantidade de processos, a fatia da imagem para cada processo fica menor, ou seja o cálculo fica mais rápido, porém o tempo de cálculo vai se aproximando do tempo de espera das trocas de mensagens.

4.5 Sugestões de melhorias ou otimizações futuras

A função apply convolution realiza os cálculos de todo o trecho passado para ela, inclusive dos pixels fantasmas. Como possíveis melhorias, podemos colocar para a iteração que percorre a altura da imagem, iniciar após a célula fantasma e parar antes da célula fantasma, pois essa parte não é utilizada na hora de concatenar a imagem. Como futura melhoria, também poderíamos implementar uma otimização mais balanceada, usando Scatterv e distribuindo de forma que um processo tome conta das linhas inferiores e superiores, fazendo com que tenha a mesma quantidade de comunicação que os processos internos.

5. CONCLUSÃO

5.1 Principais aprendizados do projeto

Através do desenvolvimento do projeto, foi possível compreender as principais funções do MPI e como utilizá-las para otimização do código sequencial. Além disso, pela necessidade de comunicação entre vizinhos,

estudamos as diferentes formas de decomposição de domínio e como estas poderiam influenciar a quantidade de comunicações entre processos e, em alguns casos, gerar desbalanceamento de carga.

5.2 Considerações sobre desempenho obtido

Os resultados obtidos podem ter sofrido influência do calor, à medida que os testes foram acontecendo o computador foi esquentando e isso pode ter feito o desempenho cair abaixo do que seria esperado na matriz 5x5. Por esse motivo, foram realizados 5 testes com os mesmos parâmetros para garantir um maior grau de exatidão dentro das possibilidades existentes.

6. REFERÊNCIAS

ORELLANA, Esbel T. Valero. *Aula 10 - MPI, Introdução e Aplicações Práticas*. Ilhéus: UESC, 2026. Slides de aula da disciplina CET107 - Processamento Paralelo.

ORELLANA, Esbel T. Valero. *Aula 11 - MPI, Comunicações Coletivas*. Ilhéus: UESC, 2026. Slides de aula da disciplina CET107 - Processamento Paralelo.

MPI: A Message-Passing Interface Standard. [s.l: s.n.]. Disponível em: <<https://www.mpi-forum.org/docs/mpi-5.0/mpi50-report.pdf>>. Acesso em: 26 maio 2026.

PRINCETON UNIVERSITY. Parallel code: Knowledge Base. [S. l.], [20--?]. Disponível em: <https://researchcomputing.princeton.edu/support/knowledge-base/parallel-code>. Acesso em: 28 maio 2026.

SCHEPKE, Claudio; LIMA, João V. F. Programação Paralela em Memória Compartilhada e Distribuída. In: ESCOLA REGIONAL DE ALTO DESEMPENHO DA REGIÃO SUL (ERAD/RS), 15., 2015, Gramado. **Minicursos** [...]. Porto Alegre: Instituto de Informática da UFRGS, 2015. Disponível em: <https://www.inf.ufrgs.br/erad2015/downloads/p/mc/mc-schepke.pdf>. Acesso em: 26 maio 2026.

DECLARAÇÃO DE TRANSPARÊNCIA E USO DE INTELIGÊNCIA ARTIFICIAL

Nós, Ana Luiza Oliveira de Freitas e Yuri Coutinho Costa, regularmente matriculados no curso de Ciência da Computação, declaramos para os devidos fins que, na elaboração do projeto intitulado Benchmark de Suavização com MPI, submetido como requisito para a disciplina DEC107 — Processamento Paralelo, adotamos as seguintes práticas em relação ao uso de ferramentas de Inteligência Artificial (IA) generativa:

() Não utilizamos ferramentas de Inteligência Artificial na concepção, desenvolvimento, programação ou redação deste trabalho.

(x) Utilizamos ferramentas de Inteligência Artificial como recursos de apoio.

As ferramentas utilizadas foram: Google Gemini. O uso se deu exclusivamente nas seguintes etapas do projeto:

1. Estruturação inicial de ideias e revisão bibliográfica;
2. Otimização da função de leitura do arquivo;
3. Auxílio na depuração (debugging) de blocos específicos de código;

Declaro estar ciente de que ferramentas de IA não são isentas de erros, vieses ou alucinações. Sendo assim, atesto que toda a responsabilidade pela autoria, integridade, originalidade e exatidão do conteúdo submetido é exclusivamente minha. Confirmando que todo o código-fonte, dados de testes e textos gerados ou modificados com o auxílio de IA foram rigorosamente revisados, testados e validados por mim, garantindo que os resultados refletem meu próprio entendimento e esforço intelectual, sem incorrer em plágio intelectual ou acadêmico.

Ilhéus - BA, 01 de Junho de 2026

Ana Luiza Oliveira de Freitas - 202310767

Yuri Coutinho Costa - 202311127